

리눅스 시스템 프로그래밍 학습 기록

남궁명수

[5] TCP 소켓과 커널 기반 네트워크 통신 구조.....	2
[01] TCP Echo Server: 클라이언트와 서버 간의 기본적인 연결 및 패킷 교환.....	4
[01-1] 서로 다른 환경의 프로세스가 어떻게 하나의 ‘연결’을 맺는가?.....	4
[01-2] 시스템 레벨 관찰: 소켓 생성부터 데이터 송수신까지의 시스템 콜 흐름.....	4
[01-3] 이론	11
[01-4] 추가	12
[02] 주소 할당과 바인딩: IP와 Port 커널 관리 체계 분석.....	15
[02-1] 이미 사용 중인 포트에 다시 서버를 띄우면 왜 에러가 나는가?.....	15
[02-2] 시스템 레벨 관찰: netstat/ss를 통한 소켓 상태 전이 모니터링.....	15
[02-3] 이론	17
[02-4] 추가	19
[03] 단순 반복문 서버의 한계와 동시성 필요성.....	22
[03-1] 왜 while 루프 하나로 구성된 서버는 두 번째 클라이언트의 접속을 받지 못하는가?.....	22
[03-2] 시스템 레벨 관찰: 멀티스레드 기반 에코 서버 구현.....	22
[03-3] 이론	26
[03-4] 추가	28
[04] 네트워크 지연 및 응답 없음 상황에서의 방어 전략.....	31
[04-1] 네트워크 지연이나 끊김 상황에서 서버가 영원히 멈추는 것을 어떻게 막는가?.....	31
[04-2] 시스템 레벨 관찰: setsockopt()를 통한 타임아웃 증명.....	31
[04-3] 이론	33
[04-4] 과제	35

[5] TCP 소켓과 커널 기반 네트워크 통신 구조

TCP와 소켓을 이해할 때 가장 중요한 전제는 “네트워크 통신도 결국 커널이 관리하는 프로세스 간 통신이다”라는 점이다. 지금까지 다뤘던 파이프 IPC가 같은 머신 내부에서 커널 버퍼를 통해 프로세스를 연결하는 구조였다면, TCP 소켓은 그 범위를 네트워크 전체로 확장한 형태라고 보면 된다. 프로세스는 직접 네트워크 하드웨어를 다루지 않고, 항상 커널이 제공하는 소켓이라는 추상화된 인터페이스를 통해서만 데이터를 읽고 쓴다.

이때 소켓은 파일 디스크립터로 표현되기 때문에 `read`와 `wrtie`라는 동일한 방식으로 다뤄지며, 이 점이 파일, 파이프, 소켓이 모두 같은 계층에서 취급되는 이유다.

결국 리눅스에서 모든 입출력은 `fd`라는 하나의 구조 위에 통합되어 있고, 커널이 그 `fd`가 어떤 대상인지에 따라 내부 동작만 다르게 처리하는 구조다.

네트워크 통신에서 프로세스를 정확히 찾아가기 위해서는 IP 주소와 포트번호가 필요하다. IP는 어떤 머신인지, 포트는 그 머신 안에서 어떤 프로세스가 통신을 담당하는지를 의미한다. 즉, 네트워크 통신은 단순히 컴퓨터 간 연결이 아니라 특정 머신의 특정 포트를 점유하고 있는 프로세스와의 연결이라고 이해해야 한다.

TCP는 이러한 연결을 단순한 데이터 전송이 아니라 “연결 상태”로 관리한다는 점이 핵심이다. 클라이언트가 `connect`를 호출하고 서버가 `listen`과 `accept`를 통해 이를 받아들이면 커널 내부에서는 3-way handshake를 통해 서로의 연결을 확정하고, 이 연결을 하나의 상태로 유지한다. 이 과정에서 커널은 해당 연결을 소켓 테이블에 등록하고 이후의 `read`와 `write`를 해당 연결에 매핑한다.

소켓은 단순한 통신 도구가 아니라 상태를 가지는 구조체이며 커널 내부에서는 상태 머신처럼 관리된다. `LISTEN` 상태에서 연결을 기다리고, 연결 요청이 들어오면 `SYN` 관련 상태를 거쳐 최종적으로 `ESTABLISHED` 상태가 된다.

이후 통신이 끝나면 바로 종료되는 것이 아니라 `TIME_WAIT` 상태로 일정 시간 유지되는데, 이는 네트워크 지연으로 인해 늦게 도착하는 패킷을 안전하게 처리하고 연결을 재사용으로 인한 충돌을 방지하기 위한 구조다.

이 때문에 서버를 재시작했을 때 동일 포트를 바로 사용할 수 없고 `Address already in use` 오류가 발생하는 현상이 생긴다.

또한 소켓 I/O는 기본적으로 blocking 방식으로 동작한다는 점이 중요하다.

`read`나 `accept`는 데이터나 연결이 들어올 때까지 프로세스를 멈춘 상태로 대기하게 되며, 이 때문에 단일 프로세스 서버는 한 번에 하나의 클라이언트만 처리할 수 있다.

동시에 여러 요청을 처리하기 위해서는 `fork`나 `pthread`를 통해 실행 흐름을 분리하거나, `epoll` 같은 이벤트 기반 구조를 사용해야 한다. 이 구조적 차이가 서버 성능과 확장성을 결정한다.

마지막으로 이 전체 구조는 보안 관점에서 매우 중요한 의미를 가진다. 소켓 통신은 외부 입력이 시스템 내부로 직접 들어오는 대표적인 경로이며, blocking 구조는 서비스 거부 공격의 주요 대상이 된다. 또한, 연결 상태와 포트 노출 자체가 공격 표면이 되기 때문에 네트워크 서버 설계에서는 항상 타임아웃, 동시 접속 제한, 상태 관리가 필수적으로

고려된다. 결국 TCP 와 소켓 구조를 이해한다는 것은 단순히 네트워크 프로그래밍을 배우는 것이 아니라 커널이 프로세스 간 통신과 상태를 어떻게 관리하는지를 이해하는 것이다.

[01] TCP Echo Server: 클라이언트와 서버 간의 기본적인 연결 및 패킷 교환

[01-1] 서로 다른 환경의 프로세스가 어떻게 하나의 '연결'을 맺는가?

- IP 주소와 포트 번호만으로 어떻게 상대방 프로세스의 정확한 문 앞까지 찾아가 데이터를 주고받는가

[01-2] 시스템 레벨 관찰: 소켓 생성부터 데이터 송수신까지의 시스템 콜 흐름

코드:

01_echo_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h> // 소켓 함수

// 소켓 프로그래밍 시 IP 주소 변환(숫자 <-> 문자열)과 바이트 순서 변환(호스트 <-> 네트워크)
#include <arpa/inet.h>

/*
socket(): 전화기 만들기 | 새전화기 사기
bind(): 전화번호 설정 | 9999번 번호 부여
listen(): 대기 상태 | 전화 올 때 기다리기
accept(): 전화 받기 | 수화기 들기
read(): 손님 말 듣기 | "여보세요 뭐라고 했지?"
write(): 그대로 말하기 | "아~ 똑같이 따라하기"
*/

/*
struct sockaddr_in {
    - sin_family(주소 체계)
    - sin_port(포트 번호)
    - sin_addr(IP 주소)
        - in_addr(-> s_addr(32비트 IP 주소 지정))
    - sin_zero(0으로 채워진 패딩)
}
*/

#define PORT 9999

int main(){
    // 서버 소켓(문지기), 클라이언트 전용 소켓(통신용)
    int serv_sock, clnt_sock;
    // 서버 주소 정보, 클라이언트 주소 정보
    struct sockaddr_in serv_addr, clnt_addr;
    socklen_t clnt_addr_size;
    char message[1024];
    int str_len;

    // --- 1. 소켓 생성 (전화기 구입) ---
    serv_sock = socket(PF_INET, SOCK_STREAM, 0);
```

```

// PF_INET(Protocol Family for Internet(IPv4))
// SOCK_STREAM: 소켓 타입이 스트림방식
// 0: 기본 프로토콜인 TCP가 자동으로 선택

// --- 2. 주소 설정(전화번호 할당) ---
memset(&serv_addr, 0, sizeof(serv_addr)); // 구조체 초기화
// memset(설정하고자하는 메모리, 설정값, 값을 채울 바이트 크기)

// 주소 체계
serv_addr.sin_family = AF_INET;
// AF_INET(IPv4): Address Family for Internet, 주소 형식을 지정할 때 사용

// IP 주소
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
// s_addr: in_addr 구조체의 멤버, IPv4 주소
// htonl: host to network long, 호스트 바이트 순서로 저장된 32비트 숫자(주소)를 네트워크 바이트(빅 엔디안)으로
변환하는 함수
// INADDR_ANY(0.0.0.0): 소켓이 컴퓨터의 모든 IP주소(NIC가 여러 개인 경우 포함)에서 들어오는 요청을 수신하도록
자동 할당
// 즉, 이 서버는 모든 네트워크 인터페이스(IP)에서 들어오는 요청을 받는다.
// 로컬에서만 할거면 "127.0.0.1"로 설정해야함

// 포트 지정
serv_addr.sin_port = htons(PORT);
// htons: host to network short(2바이트 짧은 정수), 호스트 컴퓨터의 바이트 순서(보통 리틀 엔디안)를 네트워크 표준
바이트 순서(빅 엔디안)로 변환한다.

// --- 3. 주소 할당 ---
bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr));
// bind: 소켓(serv_sock)에 주소 정보를 묶는(할당) 함수
// serv_sock: socket() 함수로 생성된 서버의 소켓 파일 디스크립터
// (struct sockaddr*): bind함수는 범용 주소 구조체인 struct sockaddr * 타입을 인자로 받음
// sizeof(serv_addr): 주소 구조체의 크기를 바이트 단위로 전달

// --- 4. 대기 상태로 전환 ---
listen(serv_sock, 5);
// listen: TCP/IP 소켓 프로그래밍에서 서버 소켓을 연결 요청 대기 상태로 전환하는 함수
// 최대 5개의 연결까지 대기 큐에 수용
// serv_sock: 서버 소켓 파일 디스크립터, 이 소켓은 이제 클라이언트의 연결을 받는 문지기 역할 수행
// 5(두번째 인자 - Backlog): 연결 요청 대기 큐, 이 서버가 동시에 처리할 수 있는 접속 대기 요청의 수, 최대 5개까지
큐에 쌓을 수 있음

// --- 5. 연결 수락(전화 받기) -> 통신용 새로운 FD 생성됨 ---
clnt_addr_size = sizeof(clnt_addr);
// 클라이언트의 주소 정보를 담을 구조체의 크기 계산

clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
// accept: 서버 소켓(serv_sock)에 대기 중인 클라이언트의 연결 요청 큐에서 첫 번째 연결 요청을 꺼내어 연결. 연결이
성공하면 클라이언트와 통신할 수 있는 새로운 소켓 파일 디스크립터를 반환
// serv_sock: bind()와 listen()을 마친 서버의 대기 소켓
// (struct sockaddr*)&clnt_addr: 연결된 클라이언트의 주소 정보가 저장될 구조체 변수의 포인터
// &clnt_addr_size: clnt_addr 구조체의 크기를 담고 있는 변수의 포인터
// clnt_sock: 반환값, 성공시에는 클라이언트와 1:1 통신을 위한 새로운 소켓 디스크립터, 실패시 -1

printf("[서버] 클라이언트 연결 성공\n");

```

```

// -- 6. 데이터 수신 및 그대로 돌려주기(Echo) ---
while((str_len = read(clnt_sock, message, sizeof(message))) != 0){
    write(clnt_sock, message, str_len);
    message[str_len] = '\0';
    printf("[서버] 받은 메시지: %s", message);
}

close(clnt_sock);
close(serv_sock);
return 0;
}

```

01_echo_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

int main(){
    int sock;
    struct sockaddr_in serv_addr;
    char message[1024];
    int str_len;

    // --- 1. 소켓 생성 ---
    sock = socket(PF_INET, SOCK_STREAM, 0);

    // --- 2. 서버 주소 설정 ---
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    serv_addr.sin_port = htons(9999);

    // --- 3. 서버에 연결 시도 ---
    if(connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1){
        // connect: 클라이언트가 서버에 연결을 요청할 때 사용하는 POSIX 표준 소켓 함수
        perror("connect() 에러");
        exit(1);
    }

    printf("[클라이언트] 서버에 연결\n");

    while(1) {
        fputs("메시지 입력(Q to quit): ", stdout);
        fgets(message, sizeof(message), stdin);

        if(!strcmp(message, "q\n") || !strcmp(message, "Q\n")) break;

        write(sock, message, strlen(message));
        str_len = read(sock, message, sizeof(message) - 1);
        message[str_len] = '\0';
        printf("[클라이언트] 서버로부터 Echo: %s", message);
    }
}

```

```
}  
  
close(sock);  
return 0;  
}
```

TCP 소켓 프로그래밍은 서버(Server)와 클라이언트(Client)가 서로 연결을 맺고 데이터를 주고 받는 구조이다.

현재 작성한 코드는 가장 기본적인 형태의 TCP Echo Server/Client 구조이며, 클라이언트가 보낸 데이터를 서버가 그대로 다시 돌려주는 방식이다.

서버는 먼저 통신을 받을 준비를 해야 한다.

이를 위해 가장 먼저 소켓을 생성한다.

```
serv_sock = sock(PF_INET, SOCK_STREAM, 0);
```

이 소켓은 실제 데이터 통신을 담당하는 소켓이라기보다, 연결 요청을 받기 위한 “대기용 소켓” 역할을 한다. 즉, 서버의 대표 전화번호 같은 역할이다.

그 다음 서버가 사용할 주소 정보를 설정한다.

```
struct sockaddr_in serv_addr;
```

sockaddr_in 구조체는 IPv4 주소 정보를 저장하기 위한 구조체이며 내부에는 다음과 같은 정보가 들어 있다.

- sin_family: 주소 체계(AF_INET = IPv4)
- sin_port: 포트 번호
- sin_addr.s_addr: IP 주소
- sin_zero: 패딩 영역

서버는 이 구조체를 이용하여 “어떤 IP와 어떤 포트에서 서비스를 할 것인지”를 지정한다.

```
serv_addr.sin_family = AF_INET;  
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);  
serv_addr.sin_port = htons(PORT);
```

INADDR_ANY 는 0.0.0.0 을 의미하며, 현재 컴퓨터의 모든 네트워크 인터페이스(IP)에서 들어오는 요청을 허용한다는 뜻이다.

즉, 서버는 특정 IP 하나만 사용하는 것이 아니라, 이 컴퓨터로 들어오는 모든 네트워크 요청을 받을 수 있게 된다.

htons()와 htonl()은 호스트 바이트 순서를 네트워크 바이트 순서(Big Endian)로 변환하는 함수이다.

- htons: short(2byte) 변환 -> 포트 번호
- htonl: long(4byte) 변환 -> IP 주소

주소 설정이 끝나면 bind()를 사용하여 소켓에 주소를 할당한다.

```
bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr));
```

이 과정은 “이 서버는 9999 번 포트를 사용한다”라고 운영체제에 등록하는 과정이다.

그 다음 listen()을 통해 연결 요청을 받을 수 있는 상태로 전환한다.

```
listen(serv_sock, 5);
```

여기서 5는 backlog queue 크기이다.

이는 “동시에 5 명과 통신 가능”이라는 의미가 아니라, 서버가 아직 처리하지 못한 연결 요청을 최대 5 개까지 대기열에 저장할 수 있다는 의미이다.

예를 들어, 서버가 현재 클라이언트 A 와 통신 중일 때, B, C, D 가 접속을 시도하면 이 요청들은 backlog queue 에 잠시 저장된다.

큐가 가득 찬 상태에서 새로운 연결 요청이 들어오면 연결이 거부될 수 있다.

이제 서버는 accept()를 통해 실제 연결을 수락한다.

```
clnt_sock = accept(serv_sock,  
                  (struct sockaddr*)&clnt_addr,  
                  &clnt_addr_size);
```

여기서 중요한 점은 accept()가 새로운 소켓을 생성한다는 것이다.

서버에는 두 종류의 소켓이 존재하게 된다.

- serv_sock
 - 연결 요청을 받는 대기용 소켓
 - 계속 listen 상태 유지
- clnt_sock
 - 특정 클라이언트와 실제 통신을 수행하는 소켓

즉, 서버는 serv_sock 으로 연결 요청만 받고, 실제 데이터 송수신은 clnt_sock 으로 수행한다.

클라이언트 측에서는 먼저 소켓을 생성한다.

```
sock = socket(PF_INET, SOCK_STREAM, 0);
```

그 다음 서버 주소를 설정한다.

```
serv_addr.sin_family = AF_INET;  
serv_addr.sin_addr.s_addr = inet_addr("127.0.0.1");  
serv_addr.sin_port = htons(9999);
```

127.0.0.1 은 localhost 를 의미하며 자기 자신의 컴퓨터를 뜻한다.

그 후 connect()를 사용하여 서버에 연결 요청을 보낸다.

```
connect(sock,  
        (struct sockaddr*)&serv_addr,  
        sizeof(serv_addr));
```

이 요청은 서버의 listen() 상태인 소켓으로 전달되고, 서버의 accept()가 이를 수락하면 연결이 완료된다.

연결이 완료되면 클라이언트의 sock 과 서버의 clnt_sock 이 서로 연결된 상태가 된다.

이후 양쪽은 모두 read()와 write()를 사용할 수 있다.

클라이언트:

```
write(sock, message, strlen(message));
```

서버:

```
read(clnt_sock, message, sizeof(message));
```

서버:

```
write(clnt_sock, message, str_len);
```

클라이언트:

```
read(sock, message, sizeof(message));
```

이처럼 TCP 소켓은 양방향(full-duplex) 통신 구조를 가진다.

즉, 하나의 연결 안에서 읽기와 쓰기를 모두 수행할 수 있다.

1. 서버가 소켓 생성
2. 서버가 IP/PORT 설정
3. bind()로 주소 등록
4. listen() 상태 진입
5. 클라이언트가 connect() 요청
6. 서버가 accept() 수행
7. 새로운 통신용 소켓(clnt_sock) 생성
8. 클라이언트와 서버가 read/write 로 데이터 송수신
9. 서버가 받은 메시지를 그대로 다시 반환

현재 구조는 한 번에 한 명의 클라이언트만 처리하는 단일 클라이언트 서버이다.

여러 클라이언트를 동시에 처리하려면 thread, fork, select, epoll 등의 기법이 추가로 필요하다.

결과:

(1) 표준 출력 결과는 아래와 같다.

./01_ehco_server.c 먼저 실행

[서버] 클라이언트 연결 성공

[서버] 받은 메시지: 안녕

./01_ehco_client.c

[클라이언트] 서버에 연결

메시지 입력(Q to quit): 안녕

[클라이언트] 서버로부터 Echo: 안녕

메시지 입력(Q to quit): q

(2) strace 결과:

서버:

```
socket() = 3
bind(3, ...)
listen(3, 5)
accept(3, ...) = 4
read(4, ...)
write(4, ...)
```

클라이언트:

```
socket() = 3
connect(3, ...)
write(3, ...)
read(3, ...)
```

[01-3] 이론

TCP/IP 4 계층과 Sockets API

TCP 소켓 프로그래밍의 핵심은 개발자가 복잡한 네트워크 하드웨어와 패킷 처리 과정을 직접 다루지 않아도 된다는 점이다. 실제 네트워크 통신에는 데이터가 전기 신호 형태로 변환되어 랜카드를 통해 이동하고, 중간 라우터를 거치며 여러 개의 패킷으로 분할하고 재조합되는 과정이 발생한다. 또한 네트워크 오류나 패킷 손실이 발생했을 때 재전송과 순서 보장까지 이루어져야 한다.

그러나 리눅스 커널은 이러한 복잡한 과정을 TCP/IP 네트워크 스택 내부에 숨기고, 개발자에게는 단순히 파일처럼 다룰 수 있는 “소켓” 인터페이스만 제공한다.

이 때문에 개발자는 파일 입출력과 유사하게 `read()`와 `write()`를 사용하여 네트워크 통신을 수행할 수 있으며, 소켓 또한 파일 디스크립터(FD)의 일종으로 관리된다.

TCP/IP 구조는 일반적으로 4 계층 모델로 설명된다.

가장 아래인 1 계층에는 실제 전기 신호와 네트워크 장치를 다루는 네트워크 인터페이스 계층이 존재하고,

2 계층에는, IP 기반의 데이터 전달을 담당하는 인터넷 계층이 위치한다.

IP 계층은 데이터를 목적지 컴퓨터까지 전달하는 역할을 수행하지만, 데이터가 정상적으로 도착했는지 여부는 보장하지 않는다.

3 계층 전송 계층에서는 TCP가 동작하며, 패킷 순서 보장, 재전송, 흐름 제어, 연결 관리 등을 담당한다.

마지막으로 4 계층인 응용 계층에서는 HTTP, FTP, SSH 같은 실제 프로그램 프로토콜이 동작한다. 소켓 프로그래밍은 이러한 여러 계층의 복잡한 구조를 추상화하여 응용 프로그램이 쉽게 네트워크 기능을 사용할 수 있도록 만든 인터페이스라고 볼 수 있다.

네트워크 통신에서 연결 대상은 IP 주소와 포트 번호의 조합으로 식별된다.

IP 주소는 데이터를 전달할 컴퓨터 자체를 식별하는 개념이며, 현실 세계의 주소 체계로 비유하면 “어느 건물인가”를 의미한다. 그러나 하나의 컴퓨터 안에는 동시에 여러 프로그램이 네트워크를 사용할 수 있기 때문에 단순히 IP 만으로는 어느 프로세스가 데이터를 받아야 하는지 구분할 수 없다.

이를 위해 사용하는 것이 포트 번호이며, 포트는 해당 컴퓨터 내부에서 특정 프로세스를 식별하는 역할을 수행한다. 즉, 네트워크 통신은 “어느 컴퓨터(IP)의 어느 프로세스(Port)”와 연결할 것인지를 지정하는 과정이다.

TCP는 연결 지향(Connection-oriented) 프로토콜이기 때문에 실제 데이터 송수신 전에 반드시 연결 수립 과정을 거친다. 클라이언트가 `connect()`를 호출하고 서버가 `accept()`를 호출하는 동안, 커널 내부에서는 3-way Handshake가 자동으로 수행된다.

이 과정에서 클라이언트는 SYN 패킷을 보내 “연결 요청”을 전달하고, 서버는 SYN-ACK 패킷으로 응답하여 “연결 가능”상태를 알린다.

마지막으로 클라이언트가 ACK 패킷을 다시 보내면 연결이 최종적으로 성립된다.

이 과정을 통해 양측 커널은 서로의 통신 상태를 ESTABLISHED 상태로 기록하고, 이후부터는 안정적인 양방향 데이터 송수신이 가능해진다.

결국 connect()와 accept()는 단순 함수 호출처럼 보이지만, 실제 내부에서는 TCP 연결을 생성하기 위한 복잡한 패킷 교환과 상태 관리가 수행되고 있는 것이다.

[01-4] 추가

(1) System Call

TCP 소켓 프로그래밍에서 사용하는 socket(), bind(), listen(), accept() 같은 함수들은 단순한 일반 함수가 아니라 대부분 운영체제 커널과 직접 상호작용하는 시스템 콜(System Call)이다. 처음에는 단순히 “네트워크 기능을 제공하는 라이브러리 함수”처럼 보이지만, 실제로는 사용자 프로그램이 커널에게 네트워크 작업을 요청하는 인터페이스 역할을 수행한다.

사용자 프로그램은 기본적으로 User Space(사용자 영역)에서 실행된다.

이 영역에서는 보안과 안정성을 위해 하드웨어나 네트워크 장치를 직접 제어할 수 없다.

따라서 네트워크 연결 생성, 파일 접근, 프로세스 생성, 메모리 관리 같은 작업은 반드시 운영체제 커널의 도움을 받아야 한다. 이때 사용자 프로그램이 커널에게 작업을 요청하는 통로가 시스템 콜이다.

소켓 프로그래밍은 이러한 시스템 콜을 기반으로 동작한다. 예를 들어, socket() 함수는 단순히 메모리 안에서 객체를 하나 만드는 수준이 아니라, 실제로 운영체제 커널에게 “TCP 통신에 사용할 소켓을 하나 생성해달라”는 요청을 보내는 것이다. 커널은 내부적으로 네트워크 자원을 할당하고, 생성된 소켓을 식별할 수 있는 파일 디스크립터를 반환한다.

리눅스에서는 소켓 역시 파일처럼 취급된다. 리눅스의 중요한 철학 중 하나는 “Everything is a file”이며, 일반 파일뿐 아니라 터미널, 파이프, 소켓 등도 모두 파일 디스크립터 기반으로 관리된다.

그래서 소켓에서도 일반 파일처럼 read()와 write()를 사용할 수 있다. 실제로는 네트워크 통신은 “소켓 파일에 데이터를 읽고 쓰는 과정”처럼 동작한다.

(2) 패킷 스니핑과 암호화의 필요성

TCP ECHO Server 와 Client 를 localhost 환경에서 실행하고 있다.

이 상태에서 tcpdump 를 사용하면 실제 네트워크를 통해 어떤 데이터가 오가는지를 직접 확인할 수 있다.

다음 명령어는 루프백 인터페이스(lo)를 통해 9999 번 포트로 오가는 패킷을 캡처하는 명령이다.

```
sudo tcpdump -i lo -A -X port 9999
```

- sudo: 네트워크 패킷 캡처는 관리자 권한이 필요하기 때문에 사용
- tcpdump: 리눅스에서 패킷을 캡처하고 분석하는 도구
- -i lo
 - loopback 인터페이스 감시
 - 즉, 자기 자신과 통신하는 패킷 확인
- -A
 - 패킷 내용을 ASCII 문자열 형태로 출력
 - 사람이 읽을 수 있게 보여줌
- X
 - 16 진수형태 출력
- port 9999: 9999 포트를 사용하는 TCP 패킷만 필터링

실제로 서버와 클라이언트가 통신 중인 상태에서 위 명령어를 실행하고 클라이언트가 문자열이나 비밀번호를 입력하면, tcpdump 화면에는 해당 데이터가 네트워크 상에서 어떻게 이동하는지가 그대로 드러난다.

즉, 사용자는 단순히 프로그램 창에 문자열을 입력했다고 생각하지만, 실제로는 그 데이터가 TCP 패킷 내부에 포함되어 운영체제 네트워크 스택을 지나가고 있으며, 같은 시스템 혹은 네트워크 상에서 패킷을 감시할 수 있는 존재는 이를 가로채서 읽을 수 있다는 뜻이다.

예를 들어, 아래와 같은 출력은 실제 TCP 패킷이 루프백 인터페이스를 통해 이동하는 장면을 보여준다.

```
07:58:53.077653 IP localhost.35802 > localhost.9999: Flags [..], ack 5, win 512 ...
```

이 로그는 localhost 의 임시 포트(35802)에서 9999 번 포트로 TCP 패킷이 전송되고 있음을 의미한다. 아래에 출력되는 16 진수 데이터와 ASCII 문자열은 실제 패킷 내부 내용이며, 암호화가 적용되지 않은 경우 사용자가 입력한 데이터가 그대로 노출될 수 있다.

이 학습에서의 핵심은 “TCP 자체는 데이터를 암호화하지 않는다”는 사실을 직접 체감하는 것이다. TCP 는 단지 안정적으로 데이터를 전달하는 역할만 수행할 뿐이며, 전송 중인 데이터의 기밀성은 보장하지 않는다. 따라서 공격자가 동일 네트워크에 접근할 수 있거나

패킷 캡처 권한을 획득한 경우, 로그인 정보나 인증 토큰 같은 민감한 데이터가 평문 상태로 그대로 유지될 위험이 존재한다.

이 때문에 실제 보안 시스템과 서비스에서는 TCP 위에 SSL/TLS 계층을 추가로 올려 데이터를 암호화한다. HTTPS 가 대표적인 예이며, 브라우저와 서버는 TLS Handshake 를 통해 암호화 키를 교환한 뒤 이후의 모든 데이터를 암호화된 형태로 전송한다. 이렇게 되면 tcpdump 나 Wireshark 로 패킷을 캡처하더라도 내부 내용은 해독 불가능한 암호문 형태로만 보이게 된다.

즉, 이번 학습은 단순히 패킷을 구경하는 것이 아니라, “네트워크를 흐르는 데이터는 기본적으로 누구에게나 노출될 수 있다”는 보안의 출발점을 이해하는 과정이다. 그리고 SSL/TLS 가 왜 현대 보안 시스템에서 필수가 되었는지를 운영체제와 네트워크 관점에서 직접 확인할 수 있다.

[02] 주소 할당과 바인딩: IP 와 Port 커널 관리 체계 분석

[02-1] 이미 사용 중인 포트에 다시 서버를 띄우면 왜 에러가 나는가?

- 서버를 켜다 켜을 때 왜 Address already in use 에러가 발생하는가?
- 커널은 종료된 포트를 왜 즉시 놓아주지 않는가?

[02-2] 시스템 레벨 관찰: netstat/ss 를 통한 소켓 상태 전이 모니터링

1. 학습 절차

- 위 코드를 실행
- 서버 터미널에서 Ctrl + C 로 강제 종료
- 즉시 다시 서버를 실행 -> 에러 확인

2. 커널 상태 모니터링

```
# -a: 모두, -n: 숫자 표시, -t: TCP, -p: 프로세스 정보
netstat -antp | grep 9999

# 또는 최신 명령어 ss 사용
ss -atn | grep 9999
```

결과 해석:

이번 학습에서는 TCP Echo Server 와 Client 를 실행한 뒤, netstat -antp 명령을 통해 커널 내부의 소켓 상태 변화를 직접 관찰하였다.

이 과정에서 TCP 연결이 단순히 “연결됨/종료됨” 수준이 아니라, 커널 내부에서 상태(State)를 가진 채 단계적으로 관리된다는 사실을 확인할 수 있었다.

처음 서버 프로그램을 실행했을 때 다음과 같은 상태가 출력되었다.

```
tcp 0 0 0.0.0.0:9999 0.0.0.0:* LISTEN
```

이 상태는 서버가 9999 번 포트를 bind 한 뒤 listen 상태로 진입했음을 의미한다.

즉, 아직 특정 클라이언트와 연결된 것은 아니지만, 커널의 소켓 테이블(Socket Table)에 “9999 번 포트는 현재 서버 프로세스가 사용 중이며 연결 요청을 기다리고 있다”는 정보가 등록된 상태이다.

여기서 0.0.0.0:9999 는 모든 네트워크 인터페이스에서 들어오는 9999 번 포트 요청을 수신하겠다는 의미를 가진다.

이후 클라이언트 프로그램이 connect 를 호출하자 상태는 다음과 같이 변화하였다.

```
tcp 0 0 127.0.0.1:9999 127.0.0.1:44218 ESTABLISHED
tcp 0 0 127.0.0.1:44218 127.0.0.1:9999 ESTABLISHED
```

이 결과는 TCP 3-way Handshake 가 성공적으로 완료되어 서버와 클라이언트 간 연결이 성립되었음을 의미한다.

여기서 중요한 점은 연결 하나가 커널 내부에서는 양방향 상태 정보로 관리된다는 것이다. 서버 입장에서는 “9999 번 포트가 44218 번 클라이언트와 연결됨”으로 기록되고, 클라이언트 입장에서는 “44218 번 임시 포트가 서버의 9999 번 포트와 연결됨”으로 기록된다.

특히, 클라이언트 측의 44218 포트는 사용자가 직접 지정한 것이 아니라 커널이 자동으로 할당한 Ephemeral Port(임시 포트)이다.

TCP 클라이언트는 connect 시 자동으로 임시 포트를 하나 부여받고, 이를 통해 서버와 연결된다. 결국 하나의 TCP 연결은 다음 네 가지 정보 조합으로 식별된다.

- 출발지 IP
- 출발지 Port
- 목적지 IP
- 목적지 Port

즉, 단순히 “9999 번 포트 연결”이 아니라, “127.0.0.1:44218 -> 127.0.0.1:9999”라는 하나의 고유 연결 세션으로 관리되는 것이다.

실제로 클라이언트에서 hi 를 입력했을 때 서버는 다음과 같이 데이터를 수신하였다.

[서버] 받은 메시지: hi

그리고 서버는 받은 문자열을 그대로 다시 write 하여 클라이언트에게 전송하였고, 클라이언트는 다음과 같이 Echo 결과를 출력하였다.

[클라이언트] 서버로부터 Echo: hi

이 과정은 TCP 소켓이 단순한 데이터 전송이 아니라, 연결된 상태를 유지한 채 양방향 스트림(Stream) 기반 통신을 수행하고 있음을 보여준다.

즉, 파이프 IPC 가 커널 내부 버퍼를 통해 같은 머신 내 프로세스를 연결했다면, TCP 소켓은 네트워크 스택을 통해 서로 다른 프로세스를 연결하는 “확장된 IPC”라고 볼 수 있다.

이후 서버를 Ctrl + C 로 강제 종료하자 netstat 상태는 다음과 같이 변화하였다.

```
tcp 0 0 127.0.0.1:9999 127.0.0.1:44218 FIN_WAIT2
tcp 1 0 127.0.0.1:44218 127.0.0.1:9999 CLOSE_WAIT
```

이 상태 변화는 TCP 연결 종료 또한 단계적으로 수행된다는 점을 보여준다.

서버 측의 FIN_WAIT2 상태는 “연결 종료 요청(FIN)은 보냈지만 상대방의 종료 응답을 아직 기다리는 상태”를 의미한다.

반면 클라이언트 측의 CLOSE_WAIT 상태는 “상대방의 종료 요청은 받았지만 아직 자신의 소켓을 완전히 닫지 않은 상태”를 의미한다.

즉, TCP 는 단순히 연결을 끊는 것이 아니라, 양측이 서로 종료 의사를 주고받으며 안전하게 연결을 해제한다. 이 과정이 필요한 이유는 아직 전송되지 않은 데이터가 남아 있을 수 있기 때문이다.

만약 강제로 즉시 연결을 제거하면 일부 패킷이 손실될 가능성이 생긴다.

또한, 이전에 관찰된 TIME_WAIT 상태는 연결 종료 후에도 커널이 일정 시간 동안 해당 포트를 유지하는 구조를 보여준다.

tcp 0 0 127.0.0.1:9999 127.0.0.1:35802 TIME_WAIT
--

이 상태는 이미 연결은 종료되었지만, 늦게 도착하는 패킷이나 재전송 패킷이 기존 연결로 잘못 인식되는 문제를 방지하기 위해 커널이 일정 시간 동안 연결 정보를 보존하는 단계이다. 따라서, 서버를 종료한 직후 동일 포트를 다시 bind 하려 하면 “Address already in use” 오류가 발생할 수 있다.

이번 학습을 통해 확인한 핵심은 TCP 소켓 통신은 단순한 문자열 송수신이 아니라, 커널 내부의 네트워크 상태 머신(State Machine) 위에서 동작하는 매우 체계적인 연결 관리 구조라는 점이다.

connect, accept, read, write 같은 함수 호출 뒤에서는 실제로 소켓 상태 전이, 패킷 송수신, 연결 유지, 종료 절차가 모두 자동으로 수행되고 있으며, netstat 은 이러한 커널 내부 상태를 외부에서 관찰할 수 있게 해주는 중요한 도구라는 사실을 확인할 수 있었다.

[02-3] 이론

sockaddr_in 구조체와 포트 점유 원리

TCP/IP 통신에서 프로세스끼리 데이터를 주고받기 위해서는 단순히 “네트워크 연결”만으로는 부족하다. 커널은 “어느 컴퓨터의 어떤 프로세스와 연결할 것인가”를 정확하게 식별해야 하며, 이를 위해 사용하는 핵심 정보가 바로 IP 주소와 Port 번호이다.

리눅스 커널은 이 정보를 내부 소켓 테이블(Socket Table)에 저장하여 네트워크 연결 상태를 관리한다.

서버 프로그램이 socket()을 호출하면 커널은 새로운 소켓(File Descriptor)을 생성한다.

하지만 이 시점에서는 아직 “어느 주소에서 대기할 것인가”가 정해지지 않은 상태이다.

이후 bind() 시스템 콜을 호출하면서 sockaddr_in 구조체를 전달하면, 커널은 해당 소켓을 특정 IP 와 Port 에 연결(bind)한다.

sockarr_in 구조체는 IPv4 통신 주소를 표현하기 위한 자료구조이며, 내부에는 다음과 같은 정보가 저장된다.

- sin_family
 - 주소 체계 지정
 - IPv4 는 AF_INET
- sin_addr
 - IP 주소 저장
 - 예: 127.0.0.1
- sin_port
 - 포트 번호 저장
 - 예: 9999

즉, 서버는 bind()를 통해 “나는 127.0.0.1 의 9999 번 포트에서 대기하겠다”라는 사실을 커널 네트워크 장부에 등록하는 것이다. 이후 다른 클라이언트가 해당 주소로 connect()를 시도하면 커널은 소켓 테이블을 조회하여 정확한 서버 프로세스에 연결 요청을 전달한다.

이 과정에서 중요한 개념이 엔디안(Endianness)이다. 컴퓨터 CPU 는 숫자를 메모리에 저장할 때 바이트 순서를 다르게 사용하는데, x86 계열 CPU 는 일반적으로 Little-Endian 방식을 사용한다. 반면, 네트워크 프로토콜(TCP/IP)은 모든 시스템이 동일하게 통신할 수 있도록 Big_Endian(Network Byte Order)을 표준으로 사용한다.

예를 들어, 포트 번호 9999 를 그대로 넣으면 CPU 메모리 순서와 네트워크 순서가 달라질 수 있기 때문에, htons() 함수를 사용하여 Host Byte Order 로 변환해야 한다.

```
addr.sin_port = htons(9999);
```

여기서 htons 는 “Host To Network Short”의 약자이며, 16 비트(short) 데이터를 네트워크 바이트 순서로 변환한다. 만약 이 변환을 하지 않으면 커널은 전혀 다른 포트 번호로 인식할 수 있다.

TCP 연결이 종료될 때는 단순히 즉시 삭제되는 것이 아니라, 커널 내부에서 여러 상태(State)를 거치게 된다. 그중 대표적인 것이 TIME_WAIT 상태이다.

TCP 는 연결 종료 시 마지막 ACK 패킷이 유실될 가능성을 고려한다.

만약 상대방이 FIN 패킷을 다시 보냈을 때 이미 소켓 정보가 완전히 제거되어 있다면 연결 상태가 꼬일 수 있다. 이를 방지하기 위해 커널은 일정 시간 동안 소켓 정보를 유지하는데, 이 상태가 바로 TIME_WAIT 이다.

실제로 `netstat -antp` 또는 `ss -antp` 명령어를 사용하면 다음과 같은 상태 변화를 확인할 수 있다.

- LISTEN: 서버가 연결 요청을 기다리는 상태
- ESTABLISHED: 서버와 클라이언트가 정상 연결된 상태
- FIN_WAIT: 연결 종료 절차 진행 중
- CLOSE_WAIT: 상대방 종료 요청을 받은 상태
- TIME_WAIT: 마지막 ACK 유실 방지를 위해 일정 시간 유지되는 상태

따라서 서버를 종료한 직후 다시 실행하면 다음과 같은 에러가 발생할 수 있다.

```
bind: Address already in use
```

이는 프로그램이 죽지 않은 것이 아니라, 커널이 아직 해당 포트를 안전하게 회수 중이라는 의미이다. 즉, 포트는 단순한 숫자가 아니라 커널이 관리하는 네트워크 자원이며, TCP의 신뢰성을 유지하기 위해 일정 시간동안 보호되는 것이다.

서버에서는 재시작 속도가 중요하기 때문에 보통 `SO_REUSEADDR` 옵션을 사용한다.

```
int opt = 1;
setsockopt(serv_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

이 옵션을 `bind()` 전에 적용하면, `TIME_WAIT` 상태에 있는 포트도 즉시 재사용할 수 있다. 이는 개발 및 운영 환경에서 매우 자주 사용되는 필수 옵션 중 하나이다.

보안 관점에서도 포트와 소켓 상태 관리는 매우 중요하다.

공격자는 `nmap` 과 같은 포트 스캐닝 도구를 사용하여 열려 있는 포트를 탐색하고, 어떤 서비스가 실행 중인지를 분석한다.

즉, 열려 있는 포트 자체가 공격 표면이 될 수 있다. 따라서 보안 제품이나 서버 시스템은 불필요한 포트를 최소화하고, 비정상적인 연결 시도나 과도한 접속 패턴을 탐지할 수 있어야 한다.

[02-4] 추가

포트 스캐닝(Port Scanning)과 은닉

네트워크에서 포트는 단순한 숫자가 아니라 외부와 통신하기 위한 “열린 문”의 역할을 한다. 서버 프로그램이 특정 포트에서 `listen()` 상태로 대기하고 있다는 것은, 외부 클라이언트가 해당 포트를 통해 시스템 내부 서비스에 접근할 수 있다는 의미이다.

따라서 보안 관점에서 포트 관리는 시스템 방어의 가장 앞단에 위치하는 핵심 요소가 된다.

공격자는 일반적으로 시스템 내부에 어떤 서비스가 실행 중인지 알지 못한다.

그래서 가장 먼저 수행하는 작업이 포트 스캐닝이다.

포트 스캐닝은 특정 IP 주소를 대상으로 수많은 포트에 연결 요청 패킷을 보내어 어떤 포트가 열려 있는지 탐색하는 과정이다. 대표적으로 `Nmap` 과 같은 도구가 사용된다.

TCP 기반 포트 스캔에서는 보통 SYN 패킷을 사용한다. 공격자는 대상 시스템의 1 번부터 65535 번까지의 포트에 SYN 패킷을 보내고, 응답 패턴을 분석하여 포트 상태를 추론한다.

- SYN-ACK 응답: 해당 포트 열려 있음(Open)
- RST 응답: 해당 포트가 닫혀 있음(Closed)
- 응답 없음: 방화벽 차단 또는 은닉 상태(Filterd/Hidden)

예를 들어, 공격자가 9999 번 포트에 SYN 패킷을 보냈을 때 서버가 SYN-ACK 로 응답하면, 공격자는 “이 시스템에서 9999 번 포트를 사용하는 서비스가 존재한다”는 사실을 알게 된다. 이후 해당 서비스의 취약점이나 버전 정보를 조사하여 실제 침투 공격으로 이어질 수 있다.

즉, 열린 포트 자체가 공격 표면이 되는 것이다.

보안 솔루션에서는 이러한 포트 스캐닝 행위를 매우 중요한 이상 행위로 간주한다.

정상적인 사용자는 일반적으로 특정 서비스 포트 하나에만 연결을 시도한다.

예를 들어, 웹 브라우저는 80 번 또는 443 번 포트에 접속하고, SSH 클라이언트는 22 번 포트에만 접근한다.

반면, 포트 스캐닝은 짧은 시간 안에 수백~수천 개 포트에 순차적으로 SYN 패킷을 전송한다는 특징이 있다.

보안 엔진은 이러한 비정상적인 접근 패턴을 실시간으로 모니터링한다.

- 매우 짧은 시간 동안 다수 포트 접근 시도
- 실패한 연결 시도의 과도한 증가
- 순차적인 포트 번호 탐색 패턴
- 여러 대상 서버에 대한 반복적인 SYN 전송

이러한 패턴이 감지되면 보안 솔루션은 해당 IP 를 공격자로 판단하고 즉시 차단(Blacklist)하거나, 방화벽 규칙을 동적으로 수정하여 패킷을 드롭한다.

일부 시스템은 IDS/IPS(Intrusion Detection/PreventionSystem)와 연동되어 자동 대응까지 수행한다.

또한 최근 보안 기술에서는 단순 차단을 넘어서 포트 은닉 기법도 연구되고 있다. 일반적인 서버는 포트가 열려 있으면 SYN-ACK 로 응답하지만, 포트 은닉 기술은 외부 스캔에 대해 아예 응답하지 않도록 만든다. 공격자는 응답이 없으면 시스템 존재 자체를 파악하기 어려워진다.

대표적인 포트 은닉 기법은 다음과 같다.

- Silent Drop
 - SYN 패킷에 대해 아무 응답도 하지 않음
 - 공격자는 방화벽인지 실제 서버인지 구분하기 어려움
- Port Knocking
 - 특정 순서의 포트 접근이 이루어져야 실제 서비스 포트를 열어주는 방식
- Port Hopping
 - 서비스 포트 번호를 주기적으로 변경하여 탐지와 추적을 어렵게 만드는 방식
- Single Packet Authorization(SPA)
 - 인증된 특수 패킷을 받은 경우에만 포트를 활성화

이러한 기술들은 공격자가 사전에 서비스 존재를 식별하기 어렵게 만들어 공격 난이도를 높인다.

결국 네트워크 보안에서 중요한 것은 단순히 “서비스를 실행하는 것”이 아니라, 어떤 포트를 열 것인지, 누가 접근 가능한지, 어떤 연결 패턴을 비정상적으로 판단할 것인지까지 포함한 전체적인 네트워크 표면 고나리이다. TCP/IP 통신 구조와 소켓 상태를 이해하는 것은 단순 서버 개발을 넘어서, 실제 보안 시스템이 공격을 탐지하고 방어하는 원리를 이해하는 기반이 된다.

[03] 단순 반복문 서버의 한계와 동시성 필요성

[03-1] 왜 while 루프 하나로 구성된 서버는 두 번째 클라이언트의 접속을 받지 못하는가?

[03-2] 시스템 레벨 관찰: 멀티스레드 기반 에코 서버 구현

코드:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <sys/wait.h>
#include <sys/types.h>

// 좀비 프로세스 방지를 위한 시그널 핸들러
void read_childproc(int sig){
    int status;
    pid_t id = waitpid(-1, &status, WNOHANG);
    // -1: 아무 자식 프로세스나 기다림
    // &status: 종료 상태 저장
    // WNOHANG: 자식이 종료되지 않았으면 블로킹하지 말고 리턴/있으면 바로 리턴

    if (id > 0) printf("[커널] 자식 프로세스 %d 회수 완료\n", id);
}

/*
struct sigaction {
    - sa_handler: 시그널을 처리할 기본적인 함수 포인터(시그널 처리기)
        - SIG_DFL(기본 동작)
        - SIG_IGN(무시)
        - 또는 사용자 정의 함수 주소를 지정

    - sa_sigaction: 상세 시그널 처리기
        - sa_flags: SA_SIGINFO가 설정되어 있을 때 사용되는 상세 시그널 처리기
        - 시그널 발생 원인, PID 등 부가 정보 받을 수 있음

    - sa_mask: 시그널 처리 중 블록할 시그널 집합
        - 시그널 핸들러가 실행되는 동안 블록(지연)시킬 시그널 집합
        - 핸들러 실행 중 특정 시그널이 다시 들어오는 것을 방지할 때 사용

    - sa_flags: 시그널 처리 동작 수정 플래그
        - SA_SIGINFO: sa_sigaction을 사용하겠다고 설정
        - SA_RESTART: 시스템 콜이 시그널에 의해 중단되었을 때 자동으로 재시작
        - SA_NODEFER: 핸들러 실행 중에 해당 시그널이 다시 들어와도 블록하지 않음
*/

int main(){
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_addr, clnt_addr;
```

```

socklen_t adr_sz;
pid_t pid;

// 좀비 프로세스 방지 설정
struct sigaction act; // 어떤 함수 실행할지, 어떤 옵션 실행할지, 실행 중 어떤 시그널 막을지 저장
act.sa_handler = read_childproc; // SIGCHLD 발생 시 read_childproc() 자동 실행
sigemptyset(&act.sa_mask); // 시그널 핸들러 실행 중에 추가로 막을 시그널 목록을 초기화(추가로 막을 시그널 없음)
act.sa_flags = 0; // 시그널 처리 옵션 - 특별한 옵션 없이 기본 동작 사용
sigaction(SIGCHLD, &act, 0);
// sigaction: 시스템 프로그래밍에서 특정 시그널을 받았을 때 프로세스가 수행할 동작을 설정하거나 변경하는 함수
// 첫 번째 인자: 동작을 설정할 시그널 번호
// act: 새로운 시그널 동작을 설정할 구조체 포인터
// 리턴값: 성공 시 0

// --- 소켓 설정 ---
serv_sock = socket(PF_INET, SOCK_STREAM, 0);
memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(9999);

if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1) return 1;
listen(serv_sock, 5);

while(1) {
    adr_sz = sizeof(clnt_addr);
    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &adr_sz);
    if(clnt_sock == -1) continue;

    // 클라이언트가 올 때마다 새로운 자식을 복제
    pid = fork();

    if(pid == 0){
        close(serv_sock); // 자식은 리스닝 소켓 필요 없음
        char buf[1024];
        int str_len;
        while((str_len = read(clnt_sock, buf, sizeof(buf))) != 0)
            write(clnt_sock, buf, str_len);

        close(clnt_sock);
        printf("[자식] 클라이언트 서비스 종료 및 변신 해제\n");
        return 0;
    } else { // 부모 프로세스
        close(clnt_sock);
    }
}
close(serv_sock);
return 0;
}

```

수행 방법:

1. 서버를 실행
2. 이전에 만들었던 client 서버를 터미널 3 개를 더 열어 각각 client 를 실행해 접속
3. 동시에 대화가 가능한지 확인
4. 서버 터미널에서 `ps -ef | grep server` 를 입력하여, 접속한 클라이언트 수만큼 자식 프로세스가 생성되었는지 확인

결과 해석:

이번 학습에서는 하나의 서버가 여러 클라이언트의 연결을 동시에 처리하기 위해 `fork()` 기반의 멀티 프로세스 구조를 사용하는 과정을 확인할 수 있었다.

서버는 `listen()` 상태에서 연결 요청을 기다리다가, 새로운 클라이언트가 접속하면 `accept()`를 통해 연결 전용 소켓(`clnt_sock`)을 생성한다.

이후 `fork()`를 호출하여 자식 프로세스를 하나 복제하고, 해당 자식 프로세스가 오직 그 클라이언트만 전담하여 데이터를 처리하게 된다.

실제로 `netstat -antp | grep 9999` 결과를 보면 다음과 같은 특징을 확인할 수 있다.

```
tcp 0 0 0.0.0.0:9999 0.0.0.0:* LISTEN 31072/./02.out
```

이 부분은 부모 프로세스가 9999 번 포트에서 지속적으로 대기(Listen) 중임을 의미한다.

0.0.0.0:9999 는 모든 네트워크 인터페이스에서 접속을 허용한다는 뜻이며, 서버는 여전히 새로운 클라이언트 연결을 받을 준비 상태를 유지하고 있다.

그 아래의 ESTABLISHED 상태들은 실제 연결된 클라이언트 세션이다.

```
tcp 0 0 127.0.0.1:9999 127.0.0.1:60104 ESTABLISHED 31091/./02.out
tcp 0 0 127.0.0.1:9999 127.0.0.1:49048 ESTABLISHED 31096/./02.out
tcp 0 0 127.0.0.1:9999 127.0.0.1:56268 ESTABLISHED 31110/./02.out
```

여기서 중요한 점은 서버 PID 가 서로 다르다는 것이다.

- 31091
- 31096
- 31110

즉, 클라이언트가 접속할 때마다 서버가 새로운 자식 프로세스를 생성했고, 각 자식 프로세스가 독립적으로 하나의 클라이언트를 담당하고 있다는 의미이다.

반면 클라이언트 측도 각각 서로 다른 PID 와 임시 포트를 사용하여 서버와 연결되어 있다.

이 구조 덕분에 여러 클라이언트가 동시에 서버와 통신하더라도 서로 영향을 주지 않는다.

예를 들어, 한 클라이언트가 `read()`에서 오래 대기하거나 매우 느리게 데이터를 보내더라도, 다른 클라이언트는 별도의 자식 프로세스에서 독립적으로 처리되기 때문에 서비스가 멈추지 않는다.

또한 코드 내부에서 부모와 자식의 역할 분리가 명확하게 이루어진 것도 중요한 관찰 포인트이다.

```
close(clnt_sock);
```

를 통해 클라이언트 전용 소켓을 닫고 다시 `accept()` 루프로 돌아간다.

즉, 부모는 오직 “새로운 연결 접수”만 담당한다.

반면 자식 프로세스는:

```
close(serv_sock);
```

를 통해 리스닝 소켓을 닫고, 자신에게 할당된 클라이언트와의 통신만 수행한다.

이러한 역할 분리는 멀티프로세스 서버 설계의 핵심이다.

부모는 계속 새로운 연결을 받아야 하므로 클라이언트 처리에 묶이면 안 되고, 자식은 특정 클라이언트 처리에만 집중해야 한다.

또 하나 중요한 부분은 좀비 프로세스 처리이다.

자식 프로세스는 클라이언트 연결이 종료되면 `return 0` 으로 종료되는데, 부모가 이를 회수하지 않으면 프로세스 테이블에 종료 정보가 남아 좀비 프로세스가 된다.

이를 방지하기 위해 코드에서는 `SIGCHLD` 시그널 핸들러를 등록하였다.

```
act.sa_handler = read_childproc;
```

```
sigaction(SIGCHLD, &act, 0);
```

자식이 종료되면 커널이 부모에게 `SIGCHLD` 시그널을 보내고, 부모는 `waitpid()`를 호출하여 종료된 자식 정보를 회수한다.

```
waitpid(-1, &status, WNOHANG);
```

여기서 `WNOHANG` 옵션은 매우 중요하다.

부모가 자식 종료를 기다리며 블로킹되지 않고 즉시 리턴하게 만들기 때문이다.

만약 이 처리를 하지 않으면 서버는 클라이언트 접속이 반복될수록 좀비 프로세스가 계속 누적되어 결국 시스템 자원을 고갈시킬 수 있다.

tcp	0	0	0.0.0.0:9999	0.0.0.0:*	LISTEN
31072/./02.out					
tcp	0	0	127.0.0.1:56268	127.0.0.1:9999	ESTABLISHED
31109/./01_clnt.out					
tcp	0	0	127.0.0.1:49048	127.0.0.1:9999	ESTABLISHED
31095/./01_clnt.out					
tcp	0	0	127.0.0.1:60104	127.0.0.1:9999	TIME_WAIT -
tcp	0	0	127.0.0.1:9999	127.0.0.1:49048	ESTABLISHED
31096/./02.out					

tcp	0	0	127.0.0.1:9999	127.0.0.1:56268	ESTABLISHED
31110/./02.out					

이번 학습은 단순한 Echo Server 를 넘어, 실제 운영 서버가 어떻게 동시성을 처리하는지 보여주는 중요한 구조를 담고 있다. 특히 `accept()` → `fork()` → 독립 처리 흐름은 오래된 UNIX 서버 설계의 핵심 모델이며, Apache 초기 버전이나 보안 솔루션의 샌드박스 구조에서도 매우 자주 사용되었다.

보안 관점에서도 멀티프로세스 모델은 중요한 장점이 있다. 각 클라이언트 처리가 서로 다른 프로세스에서 수행되기 때문에, 특정 클라이언트 처리 중 버그나 메모리 오염이 발생하더라도 전체 서버가 즉시 무너지지 않는다.

즉, 프로세스 단위의 메모리 격리가 자연스럽게 보안 안정성을 높여주는 것이다.

반면, 단점도 존재한다.

`fork()`는 프로세스를 새로 생성해야 하므로 스레드보다 생성 비용이 높고, 프로세스 간 메모리를 직접 공유할 수 없기 때문에 IPC가 필요하다.

따라서 고성능 웹 서버들은 이후 멀티스레드 또는 이벤트 기반(`epoll`) 구조로 발전하게 된다.

결국 이번 학습은 단순한 네트워크 통신이 아니라, 운영체제가 프로세스 생성, 시그널 처리, 소켓 관리, 동시성 제어를 어떻게 조합하여 실제 서버 시스템을 구성하는 지 보여주는 대표적인 사례라고 볼 수 있다.

[03-3] 이론

Multi-process vs Mutlti-thread 서버 모델 비교

클라이언트가 한 명만 접속하는 서버는 구조가 단순하다.

서버는 `accept()`로 연결을 받고, `read()`로 데이터를 읽고, `write()`로 응답을 보내면 된다.

하지만 실제 네트워크 환경에서는 동시에 수십, 수백, 수천 개의 클라이언트가 접속한다.

이때 가장 큰 문제는 Blocking 이다.

서버가 한 클라이언트와 대화하는 동안 `read()`에서 대기 상태에 빠지면, 다른 클라이언트는 연결조차 처리되지 못한다. 이를 해결하기 위해 등장한 개념이 바로 동시성 기반 서버 모델이며, 대표적으로 `fork()`기반의 멀티프로세스 서버와 `pthread` 기반의 멀티스레드 서버가 존재한다.

멀티프레스 서버는 클라이언트가 접속할 때마다 `fork()`를 호출하여 새로운 자식 프로세스를 생성한다. 부모 프로세스는 계속해서 새로운 연결을 받는 역할만 수행하고, 각각의 자식 프로세스는 자신에게 할당된 클라이언트만 전담하여 처리한다.

이 구조의 가장 큰 장점은 메모리 격리이다.

프로세스는 서로 독립된 주소 공간을 가지므로, 한 자식 프로세스에서 세그멘테이션 폴트나 메모리 손상같은 치명적인 오류가 발생하더라도 다른 자식이나 부모 프로세스에는 직접적인

영향을 주지 않는다. 즉, 클라이언트 하나가 비정상적인 패킷을 보내 특정 자식 프로세스를 죽여도 서버 전체는 계속 살아남는다.

이 특성 때문에 멀티프로세스 모델은 안정성이 매우 중요한 보안 시스템에서 선호된다. 예를 들어, 보안 솔루션에서 악성 코드 분석을 수행할 때, 의심스러운 파일 하나를 별도의 자식 프로세스에서 실행시키는 이유가 바로 이것이다. 만약 악성코드가 분석 프로세스를 크래시시키더라도 메인 엔진은 영향을 받지 않는다. 이는 “프로세스 격리를 통한 피해 최소화” 전략이라고 볼 수 있다.

하지만 멀티프로세스 모델에는 비용이 존재한다. `fork()`는 새로운 프로세스를 만들기 때문에 페이지 테이블(PT), 파일 디스크립터 테이블, 커널 자료 구조 등을 복제해야 한다. 현대 리눅스는 CoW 를 통해 실제 메모리 복사를 최소화하지만, 여전히 프로세스 생성 자체는 무거운 작업이다. 또한, 프로세스끼리 메모리를 공유하지 않으므로 데이터를 교환하려면 Pipe, Shared Memory, Socket 같은 IPC 를 사용해야 한다. 즉, 안전하지만 느리고 복잡하다.

반면, 멀티스레드 서버는 하나의 프로세스 내부에서 여러 개의 스레드를 생성한다. 각 스레드는 독립적인 실행 흐름을 가지지만, 코드 영역과 힙, 전역 변수같은 대부분의 메모리를 서로 공유한다. 따라서 새로운 연결이 들어올 때마다 스레드를 하나 생성하면 매우 빠르게 클라이언트를 처리할 수 있다. 스레드는 프로세스처럼 전체 주소 공간을 새로 만들 필요가 없고, 보통 스택만 추가로 할당하면 되기 때문에 생성 비용이 훨씬 낮다.

또한, 메모리를 공유하기 때문에 데이터 전달이 쉽다. 예를 들어, 접속자 수 카운터나 캐시 데이터를 모든 스레드가 즉시 공유할 수 있다. 웹 서버나 채팅 서버처럼 짧은 요청이 매우 빈번하게 발생하는 환경에서는 이런 구조가 높은 성능을 만들어 낸다.

그러나 멀티스레드 모델은 성능을 얻은 대신 위험성을 감수해야 한다. 가장 큰 문제는 “공유 메모리”이다. 하나의 스레드가 잘못된 포인터를 사용하거나 메모리를 오염시키면 같은 프로세스 내부의 모든 스레드가 영향을 받을 수 있다. 심한 경우 프로세스 전체가 종료되며 서버가 다운된다. 또한 여러 스레드가 동시에 같은 변수에 접근하면 Race Condition 이 발생할 수 있기 때문에 Mutex, Semaphore, RWLock 같은 동기화 기법이 필요하다.

이 동기화는 또 다른 문제를 만든다. 락(Lock)을 잘못 설계하면 Deadlock 이 발생할 수 있고, 락 경쟁이 심해지면 오히려 성능이 급격히 떨어질 수 있다. 즉, 멀티 스레드 서버는 빠르지만 설계 난이도가 훨씬 높다.

보안 관점에서도 차이가 존재한다. 멀티프로세스 서버는 한 프로세스가 탈취당해도 다른 프로세스로 쉽게 번지지 않지만, 멀티스레드 서버는 하나의 취약점이 전체 프로세스의 모든 스레드 메모리에 영향을 줄 수 있다.

예를 들어, 인증 토큰, 암호 키, 세션 정보 같은 민감한 데이터가 같은 프로세스 내부에서 공유되고 있다면, 공격자는 한 스레드를 장악하는 것만으로 다른 스레드의 데이터까지 읽을 가능성이 생긴다. 따라서, 금융, 백신, 샌드박스 같은 고보안 시스템에서는 여전히 프로세스 기반 격리를 중요하게 사용한다.

실제 현대 서버는 두 방식을 혼합하기도 한다. 예를 들어, Nginx 는 여러 워커 프로세스를 생성한 뒤 각 프로세스 내부에서 이벤트 기반 처리를 수행하며, 일부 보안 제품은 “프로세스 격리 + 내부 스레드 병렬 처리” 구조를 함께 사용한다. 즉, 안정성과 성능은 서로 트레이드오프 관계이며, 서버의 목적에 따라 설계가 달라진다.

정리하면 멀티프로세스 서버는 느리지만 강력한 격리와 안정성을 제공하며, 멀티스레드 서버는 빠르고 효율적이지만 공유 메모리로 인한 동기화 문제와 보안 위험이 존재한다. 운영체제와 서버 프로그래밍의 핵심은 결국 이 두 모델 사이에서 어떤 균형점을 선택할 것인가에 대한 문제라고 볼 수 있다.

[03-4] 추가

자원 고갈 공격(DoS)에 대비한 서버 설계

네트워크 서버에서 가장 중요한 목표 중 하나는 단순히 “빠르게 동작하는 것”이 아니라, 공격을 받는 상황에서도 계속 살아남는 것이다. 보안의 3대 요소인 기밀성, 무결성, 가용성 중에서 서버 프로그래밍은 특히 가용성과 매우 밀접하게 연결된다.

아무리 암호화가 잘되어 있고 인증 체계가 완벽하더라도, 공격자가 서버 자체를 멈춰버리면 서비스는 사실상 무력화된다.

따라서 “서버가 절대 쉽게 죽지 않도록 만드는 것” 자체를 중요한 보안 설계로 간주한다.

멀티프로세스 혹은 멀티스레드 서버 구조는 동시에 여러 클라이언트를 처리할 수 있다는 장점이 있지만, 반대로 공격자 입장에서 서버 자원을 고갈시키기 좋은 구조이기도 하다. 예를 들어, 현재 실습한 서버는 클라이언트가 접속할 때마다 `fork()`를 호출하여 새로운 자식 프로세스를 만든다. 정상 환경에서는 효율적인 동작이지만, 만약 공격자가 수천 개의 연결 요청을 매우 빠르게 보내면 어떤 일이 발생할까?

서버는 매 연결마다 새로운 프로세스를 생성하려 시도한다.

프로세스 하나가 생성될 때마다 커널은 PID 를 할당하고, 페이지 테이블과 파일 디스크립터 테이블을 준비하며, 스택과 커널 자료구조를 생성한다.

즉, 연결 하나가 단순한 “통신”이 아니라 운영체제 자원을 소비하는 행위가 되는 것이다.

공격자가 이런 연결을 수만 개까지 늘리면 결국 서버 메모리가 부족해지고, CPU 는 컨텍스트 스위칭에 대부분의 시간을 소모하게 된다. 심한 경우 시스템의 최대 프로세스

개수(pid_max) 한계에 도달하여 새로운 프로세스를 생성하지 못하게 되고, 정상 사용자조차 접속할 수 없는 상태가 된다.

이것이 대표적인 자원 고갈 기반 DoS 공격이다. 서버를 직접 해킹하지 않아도, 단순히 자원을 끝없이 소비하게 만드는 것만으로 시스템을 마비시킬 수 있다.

특히 더 위험한 공격은 “연결만 하고 아무 행동도 하지 않는 클라이언트”이다.

예를 들어, 공격자가 TCP 연결만 성립시킨 뒤 데이터를 보내지 않으면, 서버의 read()는 Blocking 상태에 빠져 해당 프로세스나 스레드가 그대로 대기하게 된다.

이런 연결이 수천 개 쌓이면 서버의 일꾼(worker)은 모두 잠들어버리고, 실제 사용자 요청을 처리할 자원이 남지 않게 된다.

이를 Slowloris 계열 공격과 유사한 관점으로 이해할 수 있다.

이러한 문제를 방어하기 위해 실제 서버는 여러 가지 보호 전략을 사용한다.

첫 번째는 최대 프로세스/스레드 제한이다. 서버가 무한정 새로운 작업자를 생성하지 못하도록 상한선을 두는 방식이다. 예를 들어, 최대 100 개의 자식 프로세스만 허용하고, 그 이상 요청이 들어오면 연결을 거부하거나 대기 큐에 넣는다.

이는 서버 전체가 무너지는 상황을 막기 위한 최소한의 안정장치이다.

두 번째는 Worker Pool(Thread Pool) 구조이다.

현대 서버는 접속할 때마다 새로운 스레드를 만드는 대신, 미리 일정 개수의 스레드를 생성해두고 재사용한다. 예를 들어, 50 개의 워커 스레드를 미리 만들어 두고, 새로운 요청이 오면 놓고 있는 워커에게 작업만 할당한다. 이렇게 하면 스레드 생성 비용을 줄일 수 있고, 동시에 생성 가능한 최대 작업 수도 자연스럽게 제한된다. 즉, 성능과 안정성을 동시에 확보할 수 있다.

세 번째는 Timeout 설정이다. 서버는 연결된 클라이언트가 일정 시간 이상 아무 데이터도 보내지 않으면 강제로 연결을 종료한다. 예를 들어, setsockopt()의 SO_RCVTIMEO 옵션이나 select/poll/epoll 기반 타이머를 활용하면, read()가 영원히 블로킹되지 않도록 제어할 수 있다. 이는 “유령 클라이언트”가 서버 자원을 붙잡고 놓지 않는 상황을 방지하는 핵심 기술이다.

실제 보안 제품에서는 여기서 더 나아가 비정상적인 연결 패턴 자체를 탐지한다. 예를 들어, 짧은 시간 안에 수천 개의 연결을 시도하거나, 연결 후 데이터를 거의 보내지 않는 IP 공격 가능성이 높다고 판단하여 자동 차단한다. 방화벽이나 IPS 는 이러한 연결 패턴을 실시간으로 분석하여 블랙리스트에 등록하거나 Rate Limit 를 적용한다.

결국 네트워크 서버 설계에서 중요한 것은 단순한 기능이 아니라, “악의적인 사용자가 시스템 자원을 어떻게 악용할 수 있는가”를 항상 고려하는 것이다.

운영체제 수준에서 프로세스, 스레드, 파일 디스크립터, 소켓 버퍼는 모두 유한한 자원이며, 공격자는 바로 이 한계를 노린다. 따라서 안정적인 서버란 단순히 빠른 서버가 아니라, 공격 상황에서도 자원을 통제하며 끝까지 살아남을 수 있는 서버라고 볼 수 있다.

[04] 네트워크 지연 및 응답 없음 상황에서의 방어 전략

[04-1] 네트워크 지연이나 끊김 상황에서 서버가 영원히 멈추는 것을 어떻게 막는가?

[04-2] 시스템 레벨 관찰: setsockopt()를 통한 타임아웃 증명

코드:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/time.h> // struct timeval을 위함

/*
초(s)/밀리초(m)/마이크로초(u)
struct timeval {
    - tv_sec: 초          1초
    - tv_usec: 마이크로초 1/1000,000,000초
}
*/

int main(){
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_addr, clnt_addr;
    socklen_t adr_sz;
    struct timeval timeout; // 타임아웃 설정을 위한 구조체

    serv_sock = socket(PF_INET, SOCK_STREAM, 0);

    // --- 1. 타임아웃 설정 (5초) ---
    timeout.tv_sec = 5;
    timeout.tv_usec = 0;

    // --- 2. 소켓 옵션 적용 (수신 타임아웃: SO_RCVTIMEO) ----
    setsockopt(serv_sock, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout, sizeof(timeout));
    /*
int setsockopt(
    int sockfd,
    int level,
    int optname,
    const void *optval,
    socklen_t optlen
);
- serv_sock: 옵션을 적용할 소켓
- SOL_SOCKET: 소켓 자체 레벨의 옵션 설정
    - SOL_SOCKET(소켓 수준), IPPROTO_TCP(TCP 수준), IPPROTO_IP(IP 수준) 등
- SO_RCVTIMEO: 수신(receive) 타임아웃 옵션
    - recv(), read() 등이 데이터를 기다리는 최대 시간 설정
- (char *)&timeout: 타임아웃 값을 전달하는 부분
- sizeof(timeout)
```

```

*/

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(9999);

bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr));
listen(serv_sock, 5);

adr_sz = sizeof(clnt_addr);
clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &adr_sz);

char buf[1024];
printf("[서버] 클라이언트 접속. 데이터를 기다림(5초 타임아웃)...Wn");

// --- 3. 클라이언트가 아무것도 안 보내면 여기서 5초 후 탈출 ---
int str_len = read(clnt_sock, buf, sizeof(buf));

// EAGAIN: Resource temporarily unavailable
// EWOULDBLOCK: 요청한 작업을 즉시 완료할 수 없으니, 나중에 다시 시도하라
// 둘다 11로 정의되어 있다.
if(str_len < 0){
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        printf("[에러] 타임아웃 발생! 상대방이 너무 오랫동안 침묵하고 있습니다.Wn");
    } else {
        perror("read 에러");
    }
} else {
    printf("[서버] 받은 데이터: %sWn", buf);
}
close(clnt_sock);
close(serv_sock);

return 0;
}

```

수행 방법:

1. 서버 실행
2. 클라이언트 실행(아무것도 입력하지 않기)
3. 결과: 서버가 정확히 5 초 후에 종료
4. strace 관찰

결과:

표준 출력 결과는 다음과 같다.

[서버] 클라이언트 접속. 데이터를 기다림(5 초 타임아웃)...

[에러] 타임아웃 발생! 상대방이 너무 오랫동안 침묵하고 있습니다.

결과 해석:

서버는 socket()을 통해 SO_RCVTIMEO 옵션을 설정하여 read()의 최대 대기 시간을 5 초로 제한하여썩. 이후 bind()와 listen()으로 9999 번 포트에서 연결을 기다렸고, 클라이언트가 접속하자 accept()을 통해 새로운 통신용 소켓(FD 4)을 생성하였다.

서버는 클라이언트의 데이터를 기다리기 위해 read(4, ...)를 호출했지만, 클라이언트가 아무 데이터도 보내지 않았기 때문에 대기 상태에 들어갔다.

그러나 5 초 타임아웃이 설정되어 있었기 때문에 커널은 무한 대기 대신 read()를 종료시키고 -1 과 EAGAIN 오류를 반환하였다.

`read(4, ...) = -1 EAGAIN (Resource temporarily unavailable)`

이를 통해 서버는 상대방이 응답하지 않는 상황을 감지하였고, “[에러] 타임아웃 발생!”메시지를 출력한 뒤 소켓을 정상적으로 닫고 종료하였다.

즉, 이번 학습은 SO_RCVTIMEO 를 이용하면 네트워크 서버가 read()에서 영원히 블로킹되지 않도록 제어할 수 있음을 보여준다.

[04-3] 이론

Socket Options 와 Keep-alive 메커니즘

소켓(Socket)은 단순히 “연결 통로”만 제공하는 것이 아니라, 커널 내부에서 다양한 동작 정책을 조절할 수 있는 여러 옵션(Socket Option)을 함께 제공한다.

개발자는 setsockopt() 시스템 콜을 사용하여 소켓의 동작 방식을 세밀하게 제어할 수 있으며, 이는 안정적인 서버 설계에서 매우 중요한 역할을 한다.

가장 대표적인 옵션 중 하나가 SO_RCVTIMEO 와 SO_SNDTIMEO 이다.

이 옵션은 데이터를 받을 때 (read, recv)와 보낼 때(write, send) 커널이 얼마나 오래 기다릴지를 결정한다.

기본적인 Blocking Socket 은 상대방이 아무 데이터도 보내지 않으면 read()에서 무한정 대기 상태에 빠진다.

하지만 타임아웃 옵션을 설정하면 커널은 지정된 시간이 지나도 데이터가 도착하지 않을 경우 시스템 콜을 강제로 종료시키고 -1 을 반환한다. 이때 errno 에는 EAGAIN 또는 EWOULDBLOCK 이 설정된다.

즉, 서버는 다음과 같은 상황을 방어할 수 있게 된다.

- 클라이언트가 연결만 하고 아무 말도 하지 않는 경우
 - 네트워크가 끊겼는데 연결 종료 패킷이 오지 않은 경우
 - 악성 사용자가 일부러 연결을 붙잡고 자원을 고갈시키려는 경우

실제로 보안 서버나 백신 에이전트 서버에서는 이러한 “무한 대기 방지”가 필수적이다. 하나의 스레드나 프로세스가 영원히 read()에 갇혀버리면 결국 서버 전체의 응답성이 급격히 떨어질 수 있기 때문이다.

또 다른 매우 중요한 옵션은 SO_REUSEADDR 이다.

TCP 연결은 종료 직후 바로 완전히 사라지지 않고 TIME_WAIT 상태에 잠시 머무르게 된다. 이는 마지막 ACK 패킷 유실 상황에 대비하여 데이터 무결성을 보장하기 위한 TCP 의 안정장치이다.

문제는 서버를 종료했다가 즉시 다시 실행하면, 이전 포트가 아직 커널 내부에 남아 있기 때문에 다음과 같은 에러가 발생할 수 있다.

```
bind error: Address already in use
```

이를 해결하기 위해 사용하는 것이 SO_REUSEADDR 옵션이다.

```
int opt = 1;
setsockopt(serv_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

이 옵션을 활성화하면 커널은 TIME_WAIT 상태의 포트라도 재사용을 허용한다.

마지막으로 SO_KEEPALIVE 는 장시간 연결 유지 상황에서 매우 중요한 역할을 한다.

TCP 연결은 물리적으로 상대방이 죽더라도 즉시 알 수 없는 경우가 많다.

예를 들어, 클라이언트 프로세스가 비정상 종료되거나, 네트워크 케이블이 뽑혔거나, 방화벽이 중간에서 연결을 끊어버려도 서버 입장에서는 한동안 “아직 연결이 살아있다”고 착각할 수 있다.

이때 SO_KEEPALIVE 를 활성화하면 커널이 주기적으로 작은 Keep-alive 패킷을 상대방에게 전송한다.

개념적으로는 다음과 같은 흐름이다.

[커널]

"오랫동안 아무 말이 없네?"

"살아있니?" → Keep-alive 패킷 전송

만약 상대방이 응답하지 않으면 커널은 해당 연결이 끊어진 것으로 판단하고 소켓을 자동 정리한다.

이를 통해 서버는 다음과 같은 문제를 방지할 수 있다.

- 죽은 클라이언트 연결이 영원히 남아 있는 상황
- 유령 세션
- 불필요한 메모리 및 FD 점유
- 장시간 방치된 좀비 네트워크 연결

결국 Socket Option 은 단순한 부가 기능이 아니라, 서버의 안정성, 성능, 보안성을 결정하는 핵심 제어 장치이다.

[04-4] 과제

안정적인 에이전트 통신 설계

엔드포인트 보안 솔루션에서 네트워크 통신은 단순한 “데이터 전달 기능”이 아니라, 시스템 전체 안정성과 직결되는 핵심 요소이다.

항상 백그라운드에서 동작하기 때문에, 통신 설계가 잘못되면 보안보다 먼저 사용자 불만이 폭발하게 된다.

예를 들어, 수천 대의 PC 에 설치된 보안 에이전트가 중앙 관리 서버와 지속적으로 통신한다고 가정해보자.

- 악성코드 탐지 결과 보고
- 정책 업데이트 수신
- 실시간 위협 정보 조회
- 로그 업로드
- 엔진 버전 동기화

이 모든 작업은 결국 소켓 통신 위에서 동작한다.

문제는 중앙 서버가 느려지거나 네트워크 상태가 불안정해졌을 때 발생한다.

만약 에이전트가 아무런 타임아웃 없이 connect(), read(), write()를 수행한다면 어떻게 될까?

에이전트는 커널 내부의 Blocking I/O 에 붙잡혀 계속 대기 상태에 빠진다.

[보안 에이전트]

서버 응답 기다리는 중...

계속 기다리는 중...

아직도 기다리는 중...

이 상황이 수천 개의 파일 검사, 실시간 감시 스레드, 업데이트 모듈과 동시에 겹치기 시작하면 결국 사용자의 PC 전체가 느려진다.

- CPU 점유 증가
- 스레드 증가
- FD 고갈
- 응답 지연
- UI 멈춤 현상
- 디스크 큐 적체

탐지율이 아무리 좋아도 시스템을 느리게 만드는 사용자는 보안 프로그램을 꺼버린다.

그렇기에 “통신 성공”보다 “실패 시 얼마나 빨리 포기하고 자원을 반환하는가”가 중요하다.

이를 위해서 사용하는 핵심 기술이 Socket Timeout 이다.

이 설정이 들어가면 서버가 응답하지 않더라도 에이전트는 영원히 대기하지 않는다.

즉, 보안 기능보다 운영체제의 가용성을 우선 보호하는 설계가 되는 것이다.